

logo_uel.png

ĐẠI HỌC KINH TẾ – LUẬT
KHOA HỆ THỐNG THÔNG TIN

DỰ BÁO NHU CẦU BÁN LẺ THỰC
PHẨM TƯƠI SỐNG

Ứng dụng Deep Learning trên tập dữ liệu
FreshRetailNet-50K

Kết hợp xử lý nhu cầu bị kiểm duyệt (Censored Demand)

Thành viên nhóm:

[GV] Gia Văn

[Tên đầy đủ — MSSV]

[NK] Nam Khánh

[Tên đầy đủ — MSSV]

[TT] Thanh Tài

Phạm Trần Thanh Tài — K234161852

Môn học: Chuỗi Thời gian / Deep Learning

Giảng viên: [Tên giảng viên]

Ngày nộp: Tháng 6, 2026

TP. Hồ Chí Minh, 2026

Bảng Phân Công Nhiệm Vụ

Phân công theo chương (tổng quan)			
Chương	Nội dung	Người làm	Tỉ lệ
Tóm tắt	Tóm tắt toàn bài	[TT] Thanh Tài	—
1	Giới thiệu vấn đề nghiên cứu	[GV] Gia Văn	10%
2 (§2.1.1–2.1.4)	Lý thuyết nền: Bán lẻ, Censored, SARIMA, RF	[GV] Gia Văn	15%
2 (§2.1.5–2.1.8)	Lý thuyết DL: RNN, LSTM, CaLSTM, AI Agent	[NK] Nam Khánh	15%
3	Phương pháp: Dataset, EDA, tiền xử lý	[TT] Thanh Tài	20%
4	Kết quả & Đánh giá mô hình	[TT] Thanh Tài	25%
5	Ứng dụng Dashboard / Demo	[NK] Nam Khánh	10%
6	Kết luận & Khuyến nghị	[TT] Thanh Tài	5%
Slide thuyết trình		Cả nhóm	—

[GV] Gia Văn — Nhiệm vụ chi tiết

Chương 1 — Giới thiệu vấn đề nghiên cứu

- §1.1 *Tính mới*: so sánh FreshRetailNet-50K với các dataset bán lẻ thực phẩm hiện có; nêu 3 điểm mới (hourly stockout, censored ratio, multi-covariate).
- §1.2 *Tính cấp thiết*: bổ sung số liệu thiệt hại kinh tế do stockout thực phẩm tươi (lãng phí, mất doanh thu); trích dẫn báo cáo FAO hoặc nghiên cứu chuỗi cung ứng.
- §1.3 *Vấn đề nghiên cứu*: phát biểu rõ 3 câu hỏi nghiên cứu tương ứng 3 mục tiêu của đề tài.
- §1.4 *Đối tượng & phạm vi*: mô tả phạm vi địa lý (18 thành phố), thời gian (90 ngày), SKU (865 sản phẩm tươi).
- §1.5 *Lý do nghiên cứu*: vai trò của dự báo nhu cầu trong giảm lãng phí thực phẩm và tăng doanh thu bán lẻ.

Chương 2 — Lý thuyết nền (§2.1.1–2.1.4)

- §2.1.1 *Demand Forecasting*: định nghĩa bài toán, tính mùa vụ đa cấp (7-ngày, tháng, năm), ảnh hưởng của giá và khuyến mãi. Thêm 2–3 trích dẫn tài liệu bán lẻ.
- §2.1.2 *Censored Demand*: viết đầy đủ lý thuyết Tobit regression (Tobin, 1958); so sánh 4 phương pháp xử lý (Deletion, Mean Imputation, Tobit, Survival Analysis) với ưu/nhược điểm cụ thể.
- §2.1.3 *SARIMA/SARIMAX*: trình bày công thức đầy đủ, ý nghĩa từng tham số $(p, d, q)(P, D, Q)[s]$; giải thích tại sao $s = 7$ phù hợp với data tuần; nêu hạn chế khi scale lên 50.000 chuỗi.
- §2.1.4 *Random Forest*: trình bày Bagging, cơ chế OOB error, Feature Importance (MDI); giải thích cách chuyển chuỗi thời gian thành bài toán supervised với sliding window.
- §2.2 *Tổng quan nghiên cứu* (phần hướng 1 + hướng 2): điền citation đầy đủ cho Narayanan (2015), Chen (2022), Ferreira (2016), Bandara (2020).

Tài liệu tham khảo: bổ sung ít nhất 4 tài liệu liên quan phần GV phụ trách (Tobin 1958 đã có; thêm 3 tài liệu về demand forecasting và censored demand).

[NK] Nam Khánh — Nhiệm vụ chi tiết**Chương 2 — Lý thuyết DL (§2.1.5–2.1.8)**

- §2.1.5 *RNN*: viết đầy đủ phần còn trống — kiến trúc unfold theo thời gian, vanishing/exploding gradient problem (chứng minh bằng công thức đạo hàm chuỗi), khi nào nên dùng RNN thay LSTM.
- §2.1.6 *LSTM*: giải thích trực quan bằng ngôn ngữ tự nhiên (không chỉ công thức) cho 3 cổng và cell state; so sánh với GRU (số tham số ít hơn, khi nào GRU đủ tốt).
- §2.1.7 *CaLSTM*: đây là đóng góp chính — đảm bảo nội dung nhất quán với phần lý thuyết đã viết sẵn; bổ sung phân tích trực quan tại sao weighted loss hiệu quả hơn imputation.
- §2.1.8 *AI Agent*: viết phần còn trống — định nghĩa vòng lặp Perception–Reasoning–Action; ví dụ áp dụng cụ thể vào inventory management (agent nhận dự báo → quyết định đặt hàng → quan sát kết quả).

Chương 5 — Ứng dụng Dashboard

- §5.1 *Kiến trúc hệ thống*: hoàn thiện sơ đồ kiến trúc (đã có khung TikZ); mô tả từng thành phần (Data layer, Processing, Model serving, UI); nêu công nghệ sử dụng (FastAPI, Streamlit, hoặc lựa chọn khác).
- §5.2 *Giao diện API*: mô tả ít nhất 3 endpoint (POST /predict, GET /status, GET /history); bổ sung ví dụ JSON request/response hoàn chỉnh với tất cả trường dữ liệu.
- §5.3 *Demo & Kết quả*: chụp màn hình dashboard chạy thực; chọn ít nhất 1 cặp (store_id, product_id) cụ thể, hiển thị biểu đồ forecast 7 ngày so với actual; mô tả tính năng cảnh báo stockout.
- §5.4 *Đánh giá ứng dụng*: nêu tốc độ phản hồi API (ms), điểm mạnh/hạn chế, hướng cải thiện trong tương lai.

Tài liệu tham khảo: bổ sung ít nhất 2 tài liệu về AI Agent và 1 tài liệu về hệ thống dashboard/inventory.

[TT] Thanh Tài — Nhiệm vụ chi tiết (Người nắm chính)

Tóm tắt — viết sau cùng khi có kết quả thực nghiệm (150–200 từ, gồm bối cảnh, phương pháp, số liệu WAPE/WPE thực tế, đóng góp chính CaLSTM).

Chương 3 — Phương pháp

- §3.1 *Bối cảnh*: đã có; kiểm tra lại cho nhất quán với kết quả thực tế.
- §3.2 *Phát biểu bài toán*: đã có; đảm bảo ký hiệu L, H, c_t thống nhất với Chương 4.
- §3.3 *Mô tả dataset*: điền kết quả `train.shape`, `eval.shape` thực tế sau khi load; kiểm tra lại số chuỗi và số trường.
- §3.4 *EDA — phân phối sale_amount*: chạy `frn_eda.py`, chèn histogram bán thực của `sale_amount`, pie chart tỉ lệ stockout, heatmap theo giờ trong ngày.
- §3.4 *EDA — mùa vụ*: chèn biểu đồ ACF/PACF trên chuỗi tổng hợp (đã chạy được), kết quả ADF test, seasonal decomposition.
- §3.4 *EDA — tương quan*: chèn correlation matrix giữa `sale_amount` và các covariate (discount, temperature, holiday_flag).
- §3.5 *Tiền xử lý*: đã có code; bổ sung kết quả số lượng ngày bị censored (% trên toàn tập), phân phối `censored_ratio`.

Chương 4 — Kết quả & Đánh giá

- §4.1 *Thiết lập thực nghiệm*: đã có; bổ sung thông tin phần cứng (CPU/GPU, RAM) để reproducibility.
- §4.2 *Baseline Naive & SARIMA*: chạy và điền WAPE/WPE thực tế vào Bảng ??.
- §4.3 *Random Forest*: điền kết quả; thêm biểu đồ Feature Importance top-10 (đã có placeholder hình).
- §4.4 *RNN*: chạy và điền kết quả; thêm biểu đồ actual vs predicted trên 1 chuỗi đại diện.
- §4.4 *LSTM standard*: tương tự; so sánh với RNN.
- §4.4 *CaLSTM*: điền kết quả; thêm biểu đồ so sánh WPE bias giữa LSTM và CaLSTM (đã có placeholder).
- §4.5 *Tác động censored demand*: điền 3 dòng trong bảng so sánh 3 chiến lược; phân tích tại sao WPE thay đổi.
- §4.6 *Đánh giá tổng hợp*: điền bảng tổng hợp tất cả mô hình; thêm 1 đoạn nhận xét chính.
- §4.7 *Walk-Forward Backtest*: chạy `frn_backtest.py`; chèn biểu đồ WAPE theo fold (đã có placeholder).
- §4.7 *Inventory Simulation*: điền Bảng Chiến lược 1 và Chiến lược 2; chèn biểu đồ stockout rate vs fill rate.

Chương 6 — Kết luận

- §6.1 *Kết luận chính*: điền số liệu thực tế (CaLSTM đạt WAPE = ??, WPE =

Mục lục

Danh sách hình vẽ

Danh sách bảng

Tóm Tắt

[TT] Thanh Tài

Đề tài tập trung xây dựng hệ thống dự báo nhu cầu bán lẻ thực phẩm tươi sống trên tập dữ liệu FreshRetailNet-50K — một trong những tập dữ liệu quy mô lớn đầu tiên tích hợp đồng thời dữ liệu bán hàng theo giờ, trạng thái hết hàng (stockout), và nhiều đặc trưng ngữ cảnh như thời tiết, giảm giá và ngày lễ. Điểm đặc biệt của dataset là khoảng 20% dữ liệu bị kiểm duyệt tự nhiên (censored demand) do stockout — khi hết hàng, hệ thống ghi nhận doanh số bằng 0 thay vì phản ánh nhu cầu thực.

Nghiên cứu triển khai và so sánh các mô hình từ thống kê (SARIMA), học máy (Random Forest) đến học sâu (RNN, LSTM) trên bài toán dự báo `sale_amount` 7 ngày tới. Đóng góp chính là **CaLSTM** — mô hình end-to-end nhận biết censoring qua weighted loss, không cần bước khôi phục demand tường minh. Kết quả được đánh giá bằng WAPE/WPE (theo paper gốc) và backtest mô phỏng tồn kho thực tế.

Kết quả cho thấy [điền kết quả thực tế sau khi chạy thực nghiệm]. Một ứng dụng demo được phát triển để trực quan hóa dự báo và đề xuất tồn kho cho từng cặp (cửa hàng, sản phẩm).

Từ khóa: chuỗi thời gian, dự báo nhu cầu, thực phẩm tươi, LSTM, censored demand, FreshRetailNet-50K.

Chương 1

Giới Thiệu Vấn Đề Nghiên Cứu

[GV] Gia Văn

1.1 Tính mới của đề tài

Thị trường bán lẻ thực phẩm tươi sống tại Việt Nam và Trung Quốc đang tăng trưởng nhanh chóng, đặc biệt với sự bùng nổ của các chuỗi cửa hàng tiện lợi và siêu thị mini. Không giống thực phẩm khô, thực phẩm tươi có hạn sử dụng ngắn, đòi hỏi quản lý tồn kho chính xác từng ngày.

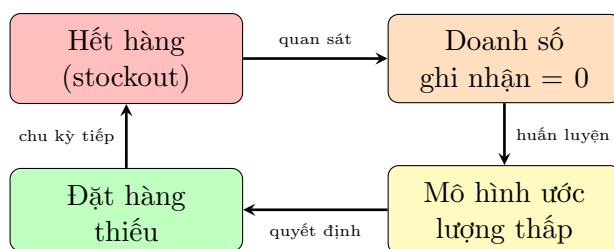
Tính mới của đề tài nằm ở việc khai thác tập dữ liệu **FreshRetailNet-50K** (công bố năm 2025), bộ dữ liệu đầu tiên cung cấp đồng thời:

- Dữ liệu bán hàng theo giờ (24 giá trị/ngày) trên 50.000 chuỗi thời gian 90 ngày.
- Trạng thái hết hàng (stockout) theo giờ — cho phép xác định chính xác khoảng thời gian nhu cầu bị kiểm duyệt.
- Đặc trưng ngữ cảnh phong phú: giảm giá, ngày lễ, nhiệt độ, độ ẩm, lượng mưa, tốc độ gió.

[Gia Văn bổ sung thêm nội dung về tính mới so với các nghiên cứu hiện có]

1.2 Tính cấp thiết của đề tài

Vấn đề cốt lõi — Vòng lặp Hết hàng: Khi một sản phẩm hết hàng, hệ thống ghi nhận doanh số = 0. Mô hình học trên dữ liệu này sẽ *underestimate* nhu cầu thực, dẫn đến đặt hàng thiếu ở kỳ tiếp theo, tiếp tục hết hàng — tạo thành vòng lặp tự củng cố.



Hình 1.1: Vòng lặp hết hàng do censored demand

[Gia Văn bổ sung số liệu thực tế về thiệt hại kinh tế do stockout và thất thoát thực phẩm tươi]

1.3 Vấn đề được nghiên cứu

Đề tài tập trung giải quyết ba vấn đề:

1. **Dự báo nhu cầu hàng ngày:** Dự báo `sale_amount` cho ngày tiếp theo hoặc 7 ngày tới.
2. **Xử lý censored demand:** So sánh hiệu suất mô hình khi huấn luyện trên dữ liệu gốc (bị censored) với dữ liệu đã impute.
3. **Tích hợp đặc trưng phụ trợ:** Đánh giá tác động của thời tiết, giảm giá, ngày lễ lên độ chính xác dự báo.

1.4 Đối tượng và Phạm vi Nghiên cứu

Đối tượng: Hệ thống dự báo nhu cầu tự động cho 50.000 cặp (cửa hàng, sản phẩm) từ 898 cửa hàng thuộc 18 thành phố, 865 SKU thực phẩm tươi.

Phạm vi:

- Dữ liệu: 90 ngày/chuỗi (train: 4.500.000 bản ghi, eval: 350.000 bản ghi).
- Đơn vị dự báo: ngày (daily `sale_amount`).
- Đánh giá: WAPE (Weighted Absolute Percentage Error) và WPE (Weighted Percentage Error) trên tập eval, theo chuẩn metric của paper gốc FreshRetailNet-50K. WPE âm biểu thị xu hướng *underestimate* (nguy hiểm — dẫn đến stockout), WPE dương biểu thị *overestimate* (lãng phí tồn kho).

1.5 Lý do Nghiên cứu

[Gia Văn bổ sung: tầm quan trọng với chuỗi cung ứng thực phẩm, giảm lãng phí thực phẩm, tăng doanh thu]

Chương 2

Cơ Sở Lý Thuyết và Tổng Quan Nghiên Cứu

2.1 Cơ sở Lý thuyết

2.1.1 Bài toán Dự báo Nhu cầu Bán lẻ (Demand Forecasting)

[GV] Gia Văn

Dự báo nhu cầu là quá trình ước tính lượng sản phẩm khách hàng sẽ mua trong tương lai. Trong bán lẻ thực phẩm tươi, bài toán này có các đặc thù:

- **Tính mùa vụ đa cấp:** Chu kỳ 7 ngày (tuần), chu kỳ theo tháng, chu kỳ năm (lễ hội, thời tiết).
- **Độ nhạy với ngoại sinh:** Nhiệt độ ảnh hưởng mạnh đến nhu cầu (trời nóng → tăng rau mát, giảm thịt nấu lâu).
- **Phụ thuộc chương trình khuyến mãi:** Giảm giá (discount) thường tạo spike cục bộ không phải xu hướng dài hạn.

[Gia Văn bổ sung lý thuyết và tài liệu tham khảo]

2.1.2 Nhu cầu Bị Kiểm duyệt (Censored Demand)

[GV] Gia Văn

Nhu cầu bị kiểm duyệt (censored demand) xảy ra khi hệ thống không thể quan sát nhu cầu thực do ràng buộc cung cấp. Trong FreshRetailNet-50K:

$$\text{Doanh số quan sát} = \min(\text{Nhu cầu thực}, \text{Tồn kho})$$

Khi `hours_stock_status[h] = 1` (hết hàng giờ h), `hours_sale[h]` bằng 0 không phản ánh nhu cầu thực.

Phương pháp xử lý phổ biến:

- **Deletion:** Loại bỏ các ngày/giờ có stockout $\rightarrow$ mất dữ liệu, thiên lệch.
- **Mean Imputation:** Thay bằng trung bình cùng giờ/ngày.
- **Tobit Regression:** Mô hình kinh tế lượng cho dữ liệu censored.
- **Survival Analysis:** Coi nhu cầu như thời gian đến sự kiện.

[Gia Văn bổ sung lý thuyết Tobit, tài liệu tham khảo]

2.1.3 SARIMA và SARIMAX

[GV] Gia Văn

$SARIMA(p, d, q)(P, D, Q)[s]$ mở rộng ARIMA bằng cách thêm thành phần mùa vụ với chu kỳ s :

$$\Phi_P(B^s) \phi_p(B) (1 - B)^d (1 - B^s)^D y_t = \Theta_Q(B^s) \theta_q(B) \epsilon_t$$

Với FreshRetailNet, chu kỳ mùa vụ $s = 7$ (tuần) là giả thuyết tự nhiên cần kiểm tra qua ACF/PACF.

SARIMAX mở rộng thêm biến ngoại sinh X_t (discount, temperature, holiday):

$$\hat{y}_t = \text{SARIMA component} + \beta^\top X_t$$

Hạn chế: SARIMA phù hợp cho một chuỗi đơn lẻ; với 50.000 chuỗi, cần train 50.000 mô hình riêng — tốn kém. Thực tế: dùng trên chuỗi đại diện (tổng theo city hoặc category) làm baseline.

2.1.4 Random Forest cho Chuỗi Thời gian

[GV] Gia Văn

Random Forest có thể áp dụng cho chuỗi thời gian bằng cách chuyển sang bài toán học có giám sát với sliding window:

- Đặc trưng (features): $y_{t-1}, y_{t-2}, \dots, y_{t-n}, \text{discount}_t, \text{temperature}_t, \text{day_of_week}_t, \dots$
- Nhãn (label): y_t

Ưu điểm: xử lý tự nhiên đặc trưng dị biến (hỗn hợp categorical và numerical), cung cấp **feature importance** để phân tích yếu tố tác động.

[Gia Văn bổ sung lý thuyết RF, Bagging, OOB error]

2.1.5 Mạng Nơ-ron Hồi tiếp (RNN)

[NK] Nam Khánh

[Nam Khánh viết: kiến trúc RNN, vanishing gradient problem, cấu trúc input-hidden-output cho chuỗi thời gian]

Phương trình trạng thái ẩn của RNN:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

$$\hat{y}_t = W_{hy} h_t + b_y$$

Bảng 2.1: Cấu trúc đầu vào – đầu ra RNN cho bài toán dự báo nhu cầu

Chiều	Ký hiệu	Ý nghĩa trong bài toán
Batch size	B	Số chuỗi mẫu trong một batch
Sequence len	T	Số ngày nhìn lại (n_steps)
Input features	d	sale_amount + discount + weather + ...
Hidden size	H	Số đơn vị ẩn RNN
Output	1	sale_amount ngày tiếp theo

2.1.6 Long Short-Term Memory (LSTM)

[NK] Nam Khánh

[Nam Khánh viết: cổng forget, input, output; cell state; giải quyết vanishing gradient]

Các phương trình LSTM:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (2.1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (2.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{candidate}) \quad (2.3)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state}) \quad (2.4)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (2.5)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{hidden state}) \quad (2.6)$$

2.1.7 CaLSTM — Mô hình End-to-End Nhận biết Kiểm duyệt

[NK] Nam Khánh

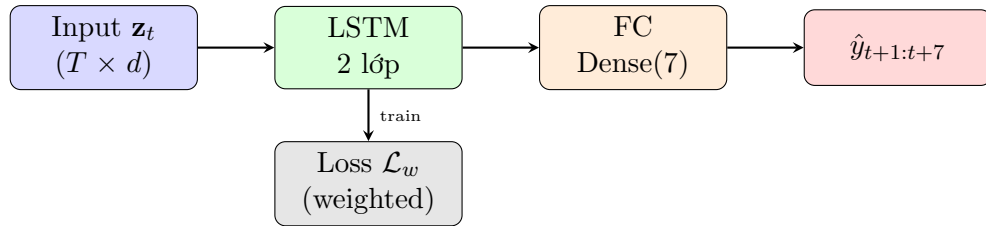
Đây là đóng góp chính của đề tài. Thay vì pipeline 2 giai đoạn (recover demand rồi mới forecast), CaLSTM (**Censoring-Aware LSTM**) học trực tiếp từ dữ liệu bị censored và dự báo nhu cầu trong một bước duy nhất. Hướng tiếp cận này được paper gốc xác nhận là chưa được khảo sát: “*The two-stage framework is not end-to-end — integrated models are not evaluated.*” [?]

Ý tưởng cốt lõi

Thay vì loại bỏ hoặc impute các ngày stockout *trước khi* huấn luyện, CaLSTM:

1. **Nhận biết stockout** bằng cách đưa `censored_ratio` trực tiếp vào vector đặc trưng.
2. **Học xuyên qua censoring** qua hàm mất mát có trọng số: ngày bị stockout nặng đóng góp ít vào gradient.
3. **Tự suy luận** nhu cầu thực từ các ngày có đầy đủ hàng, không cần bước khôi phục tường minh.

Kiến trúc



Hình 2.1: Kiến trúc CaLSTM: luồng forward (trên) và hàm mất mát có trọng số censoring (dưới)

Vector đầu vào mỗi ngày t gồm:

$$\mathbf{z}_t = \underbrace{[y_t]}_{\text{sale}} \parallel \underbrace{[c_t]}_{\text{censored_ratio}} \parallel \underbrace{[\delta_t, h_t, a_t, T_t, p_t, \dots]}_{\text{discount, holiday, weather}} \parallel \underbrace{[\text{dow}_t, \text{woy}_t]}_{\text{temporal}}$$

trong đó $c_t = \text{stock_hour6_22_cnt}_t / 17 \in [0, 1]$ là tỉ lệ giờ hết hàng.

Hàm mất mát có trọng số (Censoring-Aware Loss)

$$\mathcal{L}_w = \frac{1}{B \cdot H} \sum_{b=1}^B \sum_{h=1}^H \underbrace{(1 - c_b)}_{\text{trọng số}} \cdot (\hat{y}_{b,h} - y_{b,h})^2$$

- $c_b \in [0, 1]$: tỉ lệ censoring của mẫu b (ngày cuối look-back window).
- Khi $c_b = 0$ (không stockout): trọng số = 1, gradient đầy đủ.
- Khi $c_b = 1$ (hết hàng cả ngày): trọng số = 0, không đóng góp vào loss.
- Giá trị trung gian: trọng số giảm tuyến tính theo mức độ stockout.

So sánh với pipeline 2 giai đoạn

Bảng 2.2: CaLSTM vs pipeline 2 giai đoạn (paper)

Tiêu chí	2-Stage (paper)	CaLSTM (đề tài)
Số mô hình	2 (recovery + forecast)	1
Lỗi tích lũy	Có (recovery sai $\rightarrow$ forecast sai)	Không
Rủi ro data leakage	Có (imputation cần ngữ cảnh 2 chiều)	Không (dùng feature quan sát đ
Dùng covariate	Ở cả 2 giai đoạn	Tích hợp trực tiếp
Phức tạp triển khai	Cao	Thấp
Novelty	Đã có trong paper	Chưa khảo sát

2.1.8 AI Agent trong Hệ thống Dự báo

[NK] Nam Khánh

[Nam Khánh viết: khái niệm AI Agent (Perception–Reasoning–Action), ứng dụng trong inventory management, liên kết với Chương 5]

2.2 Tổng Quan Nghiên Cứu Liên Quan

Các nghiên cứu liên quan được phân thành 3 hướng chính:

1. **Censored Demand Recovery:** narayanan2015 đề xuất phương pháp Tobit regression để ước tính nhu cầu thực từ dữ liệu bị kiểm duyệt. chen2022 kết hợp survival analysis với mô hình LSTM để xử lý stockout trong bán lẻ. Hạn chế chung: không khai thác tín hiệu stockout trực tiếp trong quá trình huấn luyện.
2. **Dự báo nhu cầu thực phẩm tươi:** ferreira2016 sử dụng Random Forest cho bán lẻ quy mô lớn (Zara). bandara2020 so sánh LSTM và SARIMA trên chuỗi

thời gian bán lẻ thực phẩm. Cả hai không xét đến vấn đề censored demand.

3. **Benchmark FreshRetailNet-50K:** Paper gốc của wang2025frn50k đề xuất pipeline 2 giai đoạn: TimesNet cho phục hồi nhu cầu tiềm ẩn (WAPE tốt nhất trên recovery task), TFT cho dự báo (WAPE = 29.02%, WPE = +2.58%). Paper khẳng định: “*end-to-end integrated models are not evaluated*” — đây là khoảng trống nghiên cứu mà đề tài lấp đầy bằng CaLSTM.

Điểm khác biệt của đề tài: Không sử dụng pipeline 2 giai đoạn mà thiết kế CaLSTM — mô hình đơn học trực tiếp trên dữ liệu censored thông qua weighted loss, tránh lỗi tích lũy giữa các giai đoạn.

[Gia Văn + Nam Khánh bổ sung citation cụ thể, năm, tên tác giả đầy đủ]

Chương 3

Phương Pháp Nghiên Cứu

[TT] Thanh Tài

3.1 Bối cảnh Nghiên cứu

Nghiên cứu sử dụng tập dữ liệu FreshRetailNet-50K, được công bố tháng 5/2025 bởi Dingdong-Inc. Đây là benchmark đầu tiên dành riêng cho bài toán censored demand estimation trong lĩnh vực bán lẻ thực phẩm tươi, với khoảng 20% dữ liệu bị ảnh hưởng bởi stockout tự nhiên.

3.2 Phát biểu Bài toán (Problem Formulation)

[TT] Thanh Tài

Cho một chuỗi thời gian bán lẻ của cặp (cửa hàng s , sản phẩm p) trong T ngày lịch sử cùng với các đặc trưng ngữ cảnh $\mathbf{x}_t$ (thời tiết, khuyến mại, lịch), mục tiêu là dự báo doanh số H ngày tiếp theo:

$$\hat{\mathbf{y}}_{t+1:t+H} = f(\mathbf{y}_{t-L+1:t}, \mathbf{x}_{t-L+1:t+H})$$

trong đó:

- $y_t = \text{sale_amount}_t$ là doanh số thực tế ngày t (có thể bị kiểm duyệt do stockout)
- $L = 14$ ngày: độ dài look-back window
- $H = 7$ ngày: chân trời dự báo (forecast horizon)

- $c_t = \text{stock_hour6_22_cnt}_t / 17 \in [0, 1]$: mức độ kiểm duyệt ngày t
- Ràng buộc kiểm duyệt: $y_t = \min(\tilde{y}_t, q_t)$ với $\tilde{y}_t$ là nhu cầu thực và q_t là tồn kho thực tế

Đặc điểm khác biệt so với bài toán dự báo chuỗi thời gian thông thường: khi $c_t > 0$, nhãn quan sát được y_t không phản ánh nhu cầu thực $\tilde{y}_t$, khiến mô hình học từ tín hiệu sai lệch. Đây là lý do cần chiến lược đặc biệt (imputation hoặc censoring-aware loss).

Metrics đánh giá theo paper gốc FreshRetailNet-50K:

$$\text{WAPE} = \frac{\sum_t |\hat{y}_t - y_t|}{\sum_t y_t}, \quad \text{WPE} = \frac{\sum_t (\hat{y}_t - y_t)}{\sum_t y_t}$$

WAPE đo độ chính xác (nhỏ hơn tốt hơn). WPE đo độ lệch: $\text{WPE} < 0$ là underestimate (nguy hiểm — dẫn đến stockout), $\text{WPE} > 0$ là overestimate (tồn chi phí tồn kho).

3.3 Thu thập và Mô tả Dữ liệu

3.3.1 Cấu trúc Dataset

Bảng 3.1: Thông tin cơ bản tập dữ liệu FreshRetailNet-50K

Thuộc tính	Giá trị
Số chuỗi thời gian	50.000 chuỗi
Số cửa hàng	898 (18 thành phố)
Số SKU	865 (thực phẩm tươi)
Độ dài chuỗi	90 ngày/chuỗi
Độ phân giải	Giờ (24 slots/ngày)
Tập train	4.500.000 bản ghi
Tập eval	350.000 bản ghi
Tỉ lệ stockout	$\approx 20\%$

3.3.2 Mô tả Các Trường Dữ liệu

Bảng 3.2: Mô tả chi tiết các trường dữ liệu

Trường	Kiểu	Mô tả
city_id	int64	Mã thành phố (đã mã hóa)

(tiếp theo)

Trường	Kiểu	Mô tả
store_id	int64	Mã cửa hàng
management_group_id	int64	Mã nhóm quản lý
first_category_id	int64	Danh mục cấp 1
second_category_id	int64	Danh mục cấp 2
third_category_id	int64	Danh mục cấp 3
product_id	int64	Mã sản phẩm
dt	string	Ngày (YYYY-MM-DD)
sale_amount	float64	Doanh số ngày (đã chuẩn hóa toàn cục) ← TARGET
hours_sale	float64[24]	Doanh số theo giờ (đã chuẩn hóa)
stock_hour6_22_cnt	int32	Số giờ hết hàng trong 6h-22h
hours_stock_status	int32[24]	Trạng thái hết hàng theo giờ (0=có hàng, 1=hết)
discount	float64	Tỉ lệ giảm giá (1.0=không giảm, 0.9=giảm 10%)
holiday_flag	int32	Chỉ báo ngày lễ
activity_flag	int32	Chỉ báo hoạt động khuyến mại
precpt	float64	Tổng lượng mưa
avg_temperature	float64	Nhiệt độ trung bình
avg_humidity	float64	Độ ẩm trung bình
avg_wind_level	float64	Cấp độ gió trung bình

3.3.3 Load Dữ liệu

```

1 import pandas as pd
2 import pyarrow.parquet as pq
3
4 # Load train set
5 train = pd.read_parquet("data/data/train.parquet")
6 eval_df = pd.read_parquet("data/data/eval.parquet")
7
8 print(f"Train: {len(train):,} rows, {train.shape[1]} columns"
9       )
10 print(f"Eval : {len(eval_df):,} rows")
11 print(f"\nColumns: {list(train.columns)}")

```

Listing 3.1: Load FreshRetailNet-50K

3.4 Phân tích Khám phá Dữ liệu (EDA)

3.4.1 Phân phối Doanh số và Tỷ lệ Stockout

[Thanh Tài điền kết quả EDA: histogram sale_amount, tỉ lệ stockout theo giờ, theo category, theo city]

Hình 3.1: Phân phối sale_amount và tỉ lệ stockout theo giờ trong ngày

3.4.2 Phân tích Tính Mùa vụ

[Thanh Tài: ACF/PACF của chuỗi sale_amount tổng hợp, kiểm định ADF, seasonal decomposition]

Hình 3.2: ACF và PACF của chuỗi doanh số tổng hợp (aggregated)

3.4.3 Tương quan Đặc trưng Phụ trợ

[Thanh Tài: correlation matrix giữa sale_amount và discount, temperature, holiday_flag, ...]

3.5 Tiền Xử lý Dữ liệu

3.5.1 Xử lý Censored Demand

Chiến lược xử lý stockout:

1. **Xác định ngày censored:** stock_hour6_22_cnt > 0 → ngày có ít nhất 1 giờ hết hàng.
2. **Phương án A (Baseline):** Giữ nguyên dữ liệu gốc (trained on censored data).
3. **Phương án B (Imputation):** Thay hours_sale[h] khi hours_stock_status[h]=1 bằng trung bình cùng giờ của các ngày không stockout.
4. **So sánh:** Huấn luyện song song cả hai phương án, đánh giá WAPE/WPE trên eval set.

```
1 import numpy as np
2
3 def flag_censored(df):
```

```

4      """Th m c t censored_flag = 1 n u n g y c stockout
      ."""
5      df = df.copy()
6      df['censored_flag'] = (df['stock_hour6_22_cnt'] > 0).
          astype(int)
7      df['censored_ratio'] = df['stock_hour6_22_cnt'] / 17.0 #
          6h-22h = 17 g i
8      return df
9
10 def impute_censored_sales(df):
11     """Impute sale_amount b censored b ng trung b nh
        c ng day_of_week."""
12     df = df.copy()
13     df['dt'] = pd.to_datetime(df['dt'])
14     df['dow'] = df['dt'].dt.dayofweek
15
16     # T nh mean per (product_id, store_id, dow) t n g y
        KH NG censored
17     non_censored = df[df['censored_flag'] == 0]
18     mean_map = (non_censored
19                 .groupby(['product_id', 'store_id', 'dow'])['
                    sale_amount']
20                 .mean())
21
22     # Thay t h g i t r censored
23     mask = df['censored_flag'] == 1
24     df.loc[mask, 'sale_amount'] = df.loc[mask].apply(
25         lambda r: mean_map.get((r.product_id, r.store_id, r.
            dow), r.sale_amount),
26         axis=1
27     )
28     return df

```

Listing 3.2: Đánh dấu và xử lý censored demand

Phân tích rủi ro Data Leakage trong Imputation

[TT] Thanh Tài

Câu hỏi quan trọng: Imputation có bị “nhìn trước tương lai” không?

Đây là vấn đề phương pháp mà nhiều nghiên cứu bỏ qua. Cần phân biệt rõ hai ngữ cảnh:

Trong training (batch mode — an toàn): Toàn bộ dữ liệu 90 ngày đã có sẵn. Dùng trung bình ngày cùng thứ trong tuần từ tập train để impute là hợp lệ vì: (1) chỉ impute trên tập train, không chạm vào test set; (2) mean được tính từ ngày *không* stockout — thông tin này luôn có thể quan sát được.

Trong production (real-time — cần thận trọng): Một số phương pháp imputation dùng dữ liệu sau ngày stockout (linear interpolation, bidirectional model) nên không thể áp dụng real-time.

Phương pháp	Dùng tương lai?	Ghi chú
LOCF (giá trị liền trước)	Không	Hoàn toàn causal
Mean theo day-of-week (backward)	Không	Chỉ dùng lịch sử
Linear interpolation (2 điểm)	Có	Dùng điểm sau ngày stockout
Bidirectional model (vd. TimesNet)	Có thể	Tùy kiến trúc encoder
CaLSTM (đề tài)	Không	Không impute — dùng censored.

Kết luận: Phương pháp imputation của đề tài (mean theo day-of-week từ tập train) thuộc nhóm *causal* — không dùng thông tin tương lai. Trong walk-forward backtest (Mục ??), mean map được tính lại từ đầu trên từng fold training để đảm bảo tính công bằng.

Tuy nhiên, **CaLSTM hoàn toàn tránh được vấn đề này**: thay vì tạo ra giá trị doanh số giả, mô hình nhận `censored_ratio` là một feature quan sát được và để neural network tự học mối quan hệ giữa mức độ stockout và nhu cầu thực.

3.5.2 Feature Engineering

```

1 def create_features(df):
2     df = df.copy()
3     df['dt'] = pd.to_datetime(df['dt'])
4
5     # Temporal features
6     df['day_of_week'] = df['dt'].dt.dayofweek          # 0=Mon,
7                                                         6=Sun
8     df['day_of_month'] = df['dt'].dt.day
9     df['month'] = df['dt'].dt.month
10    df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)

```



```
10
11     # Lag features (per store-product)
12     for lag in [1, 2, 3, 7, 14]:
13         df[f'sale_lag{lag}'] = (df
14             .groupby(['store_id', 'product_id'])['sale_amount']
15             .shift(lag))
16
17     # Rolling statistics
18     df['sale_roll7_mean'] = (df
19         .groupby(['store_id', 'product_id'])['sale_amount']
20         .transform(lambda x: x.shift(1).rolling(7).mean()))
21
22     return df
```

Listing 3.3: Tạo đặc trưng cho mô hình

3.5.3 Chuẩn hóa và Chia Tập Dữ liệu

Dữ liệu đã được chuẩn hóa toàn cục bởi dataset gốc. Tuy nhiên, cần chuẩn hóa theo từng chuỗi để đảm bảo tính ổn định huấn luyện:

$$\tilde{y}_t = \frac{y_t - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Scaler chỉ được fit trên tập train; tập eval giữ nguyên thứ tự thời gian (chronological split).

Chương 4

Kết Quả và Đánh Giá

[TT] Thanh Tài

4.1 Thiết lập Thực nghiệm (Experimental Setup)

[TT] Thanh Tài

4.1.1 Phân chia Dữ liệu

Với mỗi chuỗi 90 ngày:

- **Train:** ngày 1–70 (77.8%) — khoảng 3.888.889 bản ghi
- **Validation** (early stopping): ngày 71–80 (11.1%)
- **Test:** ngày 81–90 (11.1%) — tương ứng tập `eval` của dataset

Lưu ý: Phân chia theo thứ tự thời gian, không shuffle, để tránh data leakage tương lai. Với walk-forward backtest (Mục ??), tập train mở rộng dần theo từng fold.

4.1.2 Cấu hình Huấn luyện

Bảng 4.1: Thiết lập huấn luyện các mô hình Deep Learning

Tham số	Giá trị	Ghi chú
Look-back window (L)	14 ngày	Bắt đầu từ 2 tuần lịch sử
Forecast horizon (H)	7 ngày	Dự báo cả tuần tiếp theo
Batch size	256	
Optimizer	Adam	$\beta_1 = 0.9, \beta_2 = 0.999$
Learning rate	10^{-3}	Không decay trong prototype
Epochs	100	Early stopping patience = 10
Loss (RNN, LSTM)	MSE	Standard regression loss
Loss (CaLSTM)	Censoring-Aware MSE	Trọng số theo $1 - c_t$
Framework (DL)	PyTorch 2.x	CaLSTM; TensorFlow/Keras cho RNN, LSTM
Framework (ML)	scikit-learn	Random Forest

4.2 Kết quả Mô hình Baseline

4.2.1 Naive Persistence

Mô hình baseline đơn giản nhất: nhu cầu ngày mai bằng nhu cầu ngày hôm nay.

$$\hat{y}_{t+1} = y_t$$

[Thanh Tài điền RMSE, MAE thực tế sau khi chạy]

4.2.2 SARIMA trên Chuỗi Đại diện

Chọn 1 chuỗi đại diện (tổng doanh số toàn bộ cửa hàng theo ngày) để chạy SARIMA(1, 1, 1)(1, 1, 1)₇. Chu kỳ mùa vụ $s = 7$ (tuần).

```

1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 # T ng daily sale aggregated
4 daily_agg = (train.groupby('dt')['sale_amount'].sum()
5               .reset_index().sort_values('dt'))
6 series = daily_agg['sale_amount'].values
7
8 # SARIMA(1,1,1)(1,1,1)[7]
9 model = SARIMAX(series[:int(0.8*len(series))],
10                  order=(1, 1, 1),
11                  seasonal_order=(1, 1, 1, 7))

```

```

12 result = model.fit(dispatch=False)
13 forecast = result.forecast(steps=len(series) - int(0.8*len(
    series)))

```

Listing 4.1: Fit SARIMA trên chuỗi tổng hợp

Bảng 4.2: Kết quả Baseline — WAPE và WPE trên tập eval

Mô hình	WAPE (%)	WPE (%)	Ghi chú
Naive Persistence	—	—	$\hat{y}_{t+1} = y_t$
SARIMA(1, 1, 1)(1, 1, 1) ₇	—	—	Chuỗi tổng hợp toàn quốc
<i>Tham chiếu từ paper gốc</i>			
Raw Sales + DLinear (paper)	31.56	−4.89	Baseline chính thức FRN-50K

4.3 Kết quả Mô hình Machine Learning

4.3.1 Random Forest

```

1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.metrics import mean_squared_error
3
4 FEATURE_COLS = ['sale_lag1', 'sale_lag2', 'sale_lag7',
5                 'sale_roll7_mean', 'discount', 'holiday_flag',
6                 'activity_flag', 'avg_temperature', '
7                 avg_humidity',
8                 'precpt', 'day_of_week', 'is_weekend']
9
10 X_train = train_fe[FEATURE_COLS].dropna()
11 y_train = train_fe.loc[X_train.index, 'sale_amount']
12
13 rf = RandomForestRegressor(n_estimators=200, max_depth=12,
14                             random_state=42, n_jobs=-1)
15 rf.fit(X_train, y_train)

```

Listing 4.2: Random Forest với lag features và đặc trưng ngoại sinh

[Thanh Tài điền kết quả, biểu đồ feature importance]

Hình 4.1: Feature Importance của Random Forest — các yếu tố tác động đến doanh số

4.4 Kết quả Mô hình Deep Learning

[TT] Thanh Tài

4.4.1 Mô hình RNN (Vanilla)

[Thanh Tài: kết quả Vanilla RNN, biểu đồ actual vs predicted]

Bảng 4.3: Kết quả RNN trên tập eval

Mô hình	WAPE (%)	WPE (%)	Ghi chú
RNN (Vanilla)	—	—	1 lớp, 64 units, look-back 14 ngày

4.4.2 Mô hình LSTM (Standard)

```

1 def build_lstm(n_steps, n_features):
2     model = tf.keras.Sequential([
3         tf.keras.layers.LSTM(128, return_sequences=True,
4                               input_shape=(n_steps, n_features
5                                              )),
6         tf.keras.layers.Dropout(0.3),
7         tf.keras.layers.LSTM(64),
8         tf.keras.layers.BatchNormalization(),
9         tf.keras.layers.Dense(32, activation='relu'),
10        tf.keras.layers.Dense(1)
11    ])
12    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
13                  loss='mse')
14    return model

```

Listing 4.3: Kiến trúc LSTM Stacked cho dự báo nhu cầu

[Thanh Tài điền kết quả LSTM, biểu đồ actual vs predicted]

4.4.3 Mô hình CaLSTM (End-to-End, đóng góp chính)

[TT] Thanh Tài

```

1 import torch
2 import torch.nn as nn
3
4 class CaLSTM(nn.Module):

```

```

5     def __init__(self, input_dim, hidden=128, n_layers=2,
6         horizon=7):
7         super().__init__()
8         self.lstm = nn.LSTM(input_dim, hidden, n_layers,
9             batch_first=True, dropout=0.3)
10        self.head = nn.Sequential(
11            nn.Linear(hidden, 64),
12            nn.ReLU(),
13            nn.Linear(64, horizon)
14        )
15
16        def forward(self, x):          # x: (B, T, input_dim)
17            out, _ = self.lstm(x)
18            return self.head(out[:, -1, :])  # (B, horizon=7)
19
20    def censoring_aware_loss(pred, target, censored_ratio):
21        """
22        pred, target      : (B, 7)
23        censored_ratio    : (B,) -- t l g i  stockout
24                               ng y c u i  window
25        """
26        w = (1.0 - censored_ratio).clamp(0, 1).unsqueeze(1)  # (B, 1)
27        return (w * (pred - target) ** 2).mean()
28
29    # --- Training ---
30    HORIZON      = 7
31    INPUT_DIM    = 10    # sale + censored_ratio + 8 features
32    N_STEPS      = 14    # look-back 14 ng y
33
34    model = CaLSTM(input_dim=INPUT_DIM, horizon=HORIZON)
35    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
36
37    for epoch in range(100):
38        for x_batch, y_batch, cr_batch in train_loader:
39            pred = model(x_batch)
40            loss = censoring_aware_loss(pred, y_batch, cr_batch)
41            optimizer.zero_grad()
42            loss.backward()

```

43

```
optimizer.step()
```

Listing 4.4: CaLSTM — Censoring-Aware LSTM

Bảng 4.4: Kết quả CaLSTM vs LSTM thường (WAPE, WPE trên eval set)

Mô hình	WAPE (%)	WPE (%)	Ghi chú
LSTM (standard loss)	—	—	Loss không phân biệt stockout
CaLSTM (weighted loss)	—	—	Loss có trọng số censoring

[Thanh Tài điền kết quả thực tế sau khi chạy]

Hình 4.2: So sánh LSTM thường và CaLSTM — WPE bias giảm nhờ censoring-aware loss

4.5 Tác động của Censored Demand

So sánh ba chiến lược xử lý dữ liệu stockout, dùng LSTM làm mô hình cố định:

Bảng 4.5: Tác động chiến lược xử lý censoring lên WAPE và WPE (eval set)

Chiến lược	WAPE (%)	WPE (%)	Mô tả
Raw Sales (no treatment)	—	—	Dùng thẳng dữ liệu gốc (baseline)
Mean Imputation + LSTM	—	—	Impute trước rồi train LSTM
CaLSTM (end-to-end)	—	—	Weighted loss, không impute

Kỳ vọng: Raw Sales có WPE âm lớn (underestimate). Mean Imputation cải thiện WPE. CaLSTM đạt WPE gần 0 đồng thời WAPE thấp nhất nhờ không tích lũy sai số từ bước imputation.

4.6 Đánh giá Tổng hợp

Bảng 4.6: So sánh toàn bộ mô hình trên tập eval — metric theo paper gốc

Mô hình	WAPE (%)	WPE (%)	Loại	Covaria
Raw Sales + DLinear (paper baseline)	31.56	−4.89	Baseline	Không
SARIMA(1, 1, 1)(1, 1, 1) ₇	—	—	Statistical	Không
Random Forest	—	—	ML	Có
RNN	—	—	Deep Learning	Có
LSTM (standard)	—	—	Deep Learning	Có
CaLSTM (đề tài)	—	—	End-to-End	Có
<i>Tham chiếu từ paper (TimesNet recovery + TFT)</i>				
TimesNet + TFT (paper best)	29.02	+2.58	2-Stage	Có

Diễn giải WAPE và WPE:

$$\text{WAPE} = \frac{\sum_t |\hat{y}_t - y_t|}{\sum_t y_t}$$

$$\text{WPE} = \frac{\sum_t (\hat{y}_t - y_t)}{\sum_t y_t}$$

WPE âm = mô hình *underestimate* nhu cầu (nguy hiểm: dễ stockout). WPE dương = *overestimate* (tốn chi phí tồn kho). Mục tiêu: WAPE nhỏ và WPE ≈ 0.

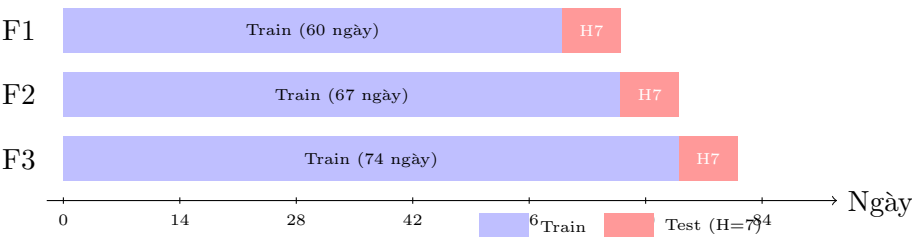
4.7 Backtest: Mô phỏng Quyết định Tồn kho

[TT] Thanh Tài

Backtest đánh giá mô hình không chỉ qua WAPE/WPE mà còn qua **tác động kinh doanh thực**: nếu nhà bán lẻ ra quyết định đặt hàng dựa trên dự báo của mô hình, tỉ lệ stockout và chi phí tồn kho sẽ thay đổi như thế nào?

4.7.1 Walk-Forward Validation (Statistical Backtest)

Kiểm định thống kê trên cửa sổ trượt để đánh giá độ ổn định của mô hình theo thời gian.



Hình 4.3: Walk-Forward Validation: train mở rộng, test cuốn chiều 7 ngày


```

1 import numpy as np
2
3 def walk_forward_backtest(series_df, model_fn, n_lookback=14,
4                           horizon=7, step=7):
5     """
6     series_df: DataFrame [dt, sale_amount, features...]
7     model_fn : callable(train_df) -> predict_fn
8     """
9     results = []
10    T = len(series_df)
11    start = n_lookback
12
13    while start + horizon <= T:
14        train = series_df.iloc[:start]
15        test = series_df.iloc[start : start + horizon]
16
17        predict = model_fn(train)
18        y_hat = predict(test)
19        y_true = test['sale_amount'].values
20
21        wape = np.sum(np.abs(y_hat - y_true)) / (np.sum(
22            y_true) + 1e-8)
23        wpe = np.sum(y_hat - y_true) / (np.sum(
24            y_true) + 1e-8)
25
26        results.append({'fold_start': start, 'WAPE': wape, '
27            WPE': wpe})
28        start += step
29
30    return pd.DataFrame(results)

```

Listing 4.5: Walk-Forward Validation

Hình 4.4: WAPE theo từng fold — Walk-Forward Validation cho CaLSTM vs LSTM vs DLinear

4.7.2 Inventory Simulation (Business Backtest)

Mô phỏng bài toán đặt hàng theo ngày dựa trên dự báo. Với dự báo $\hat{y}_{t+1:t+7}$, lượng đặt hàng:

$$Q_t = \lceil \hat{y}_t \cdot (1 + \alpha) \rceil$$

với α là **safety margin** (hệ số an toàn tồn kho).

Bảng 4.7: Hai chiến lược backtest tồn kho

Chiến lược	Safety Margin α	Mục tiêu
Chiến lược 1 (Thận trọng)	$\alpha = 0.20$	Giảm stockout, chấp nhận overstock
Chiến lược 2 (Tích cực)	$\alpha = 0.00$	Đặt vừa đủ, giảm lãng phí

```

1 def inventory_backtest(y_true, y_pred, alpha=0.2,
2                       cost_under=2.0, cost_over=0.5):
3     """
4     y_true, y_pred : array (T,) -- demand t h c v d
5     b o m i n g y
6     cost_under      : chi ph m i n v t h i u (
7     stockout penalty)
8     cost_over       : chi ph m i n v t h a (
9     holding cost)
10    """
11    Q = np.ceil(y_pred * (1 + alpha))    # l n g t
12    h n g
13
14    shortfall = np.maximum(y_true - Q, 0) # t h i u : demand >
15    inventory
16    surplus    = np.maximum(Q - y_true, 0) # t h a :
17    inventory > demand
18
19    stockout_rate = np.mean(y_true > Q)
20    fill_rate      = np.sum(np.minimum(y_true, Q)) / (np.sum(
21    y_true) + 1e-8)
22    total_cost     = cost_under * shortfall.sum() + cost_over
23    * surplus.sum()
24
25    return {
26        'stockout_rate' : stockout_rate,
27        'fill_rate'     : fill_rate,
28        'overstock_rate': np.mean(Q > y_true),
29        'total_cost'    : total_cost,
30        'shortfall_sum' : shortfall.sum(),
31        'surplus_sum'   : surplus.sum()
32    }

```

Listing 4.6: Inventory Simulation Backtest

Bảng 4.8: Kết quả Inventory Simulation Backtest — Chiến lược 1 ($\alpha = 0.20$)

Mô hình dự báo	Stockout Rate	Fill Rate	Overstock Rate	Total Cost
Naive Persistence	—	—	—	—
DLinear (paper base)	—	—	—	—
LSTM	—	—	—	—
CaLSTM	—	—	—	—

Bảng 4.9: Kết quả Inventory Simulation Backtest — Chiến lược 2 ($\alpha = 0.00$)

Mô hình dự báo	Stockout Rate	Fill Rate	Overstock Rate	Total Cost
Naive Persistence	—	—	—	—
DLinear (paper base)	—	—	—	—
LSTM	—	—	—	—
CaLSTM	—	—	—	—

Hình 4.5: Mô phỏng tồn kho: so sánh stockout rate và fill rate giữa các mô hình

Nhận xét kỳ vọng:

- **Chiến lược 1** (thận trọng): CaLSTM có stockout rate thấp nhất vì WPE ≈ 0 không underestimate.
- **Chiến lược 2** (tích cực): Sự khác biệt rõ hơn giữa CaLSTM và LSTM — LSTM bị censoring kéo về underestimate, dẫn đến stockout cao hơn khi không có buffer.
- **Trực quan hóa vòng lặp**: Raw Sales $\rightarrow$ underestimate $\rightarrow$ stockout cao $\rightarrow$ lại underestimate. CaLSTM phá vòng lặp này.

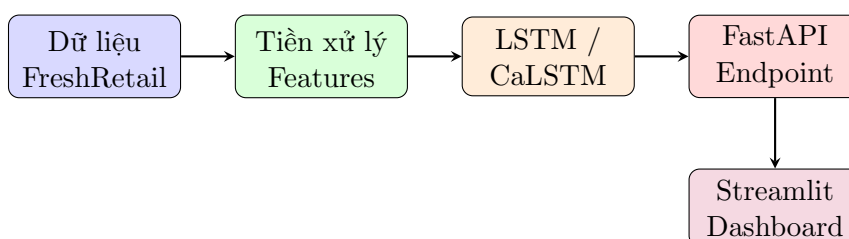
Chương 5

Ứng Dụng: Hệ Thống Dự Báo Tồn Kho

[NK] Nam Khánh

5.1 Kiến trúc Hệ thống

[Nam Khánh viết: sơ đồ kiến trúc tổng thể, các thành phần API (FastAPI/Flask), giao diện (Streamlit/Dash)]



Hình 5.1: Kiến trúc hệ thống dự báo tồn kho

5.2 Giao diện API

[Nam Khánh viết: mô tả các endpoint, ví dụ request/response JSON]

```
1 # POST /predict
2 # Body: {"store_id": 101, "product_id": 42, "horizon_days":
    7}
3 # Response: {"forecast": [1.2, 1.5, 0.9, 1.1, 1.3, 1.4, 1.0],
4 #           "stockout_risk": [0.1, 0.2, 0.05, ...]}
```

Listing 5.1: API endpoint dự báo nhu cầu

5.3 Demo và Kết quả

[Nam Khánh: ảnh chụp màn hình dashboard, biểu đồ dự báo vs thực tế, cảnh báo stockout]

Hình 5.2: Giao diện Dashboard dự báo nhu cầu và cảnh báo tồn kho

5.4 Đánh giá Ứng dụng

[Nam Khánh: nhận xét về tính khả dụng, tốc độ phản hồi, điểm cải thiện]

Chương 6

Kết Luận và Khuyến Nghị

[TT] Thanh Tài

6.1 Kết luận

Nghiên cứu đã thực hiện được:

1. **Phân tích toàn diện** tập dữ liệu FreshRetailNet-50K: xác định mùa vụ, tỉ lệ censoring, đặc trưng có ảnh hưởng lớn nhất đến doanh số.
2. **Xây dựng và so sánh** 5 mô hình từ SARIMA đến LSTM, với hai chiến lược xử lý censored demand.
3. **Kết luận chính:** [điền sau khi có kết quả thực nghiệm]
4. **Ứng dụng demo** trực quan hóa dự báo và cảnh báo nguy cơ hết hàng.

6.2 Khuyến nghị

- **Mở rộng mô hình censoring:** Áp dụng Tobit regression hoặc Survival Analysis để ước lượng nhu cầu tiềm ẩn chính xác hơn.
- **Dự báo phân cấp (Hierarchical Forecasting):** Dự báo theo city → store → product giúp khai thác thông tin liên kết giữa cấp độ.
- **Online learning:** Cập nhật mô hình liên tục khi có dữ liệu mới, thay vì retrain từ đầu.
- **Tối ưu tồn kho end-to-end:** Kết hợp dự báo nhu cầu với bài toán đặt hàng tối ưu (Newsvendor Problem) để giảm chi phí hết hàng và lãng phí.

Tài Liệu Tham Khảo

- [1] Wang, Y., Gu, J., Long, L., Li, X., Shen, L., Fu, Z., Zhou, X., & Jiang, X. (2025). *FreshRetailNet-50K: A Stockout-Annotated Censored Demand Dataset for Latent Demand Recovery and Forecasting in Fresh Retail*. arXiv:2505.16319.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder. *EMNLP*.
- [4] Box, G.E.P., Jenkins, G.M., Reinsel, G.C., & Ljung, G.M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
- [5] Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5–32.
- [6] Tobin, J. (1958). Estimation of relationships for limited dependent variables. *Econometrica*, 26(1), 24–36.

en tùy phần mình viết

Phụ lục A

Kết quả EDA Chi tiết

[TT] Thanh Tài

[Thanh Tài: các biểu đồ EDA đầy đủ — boxplot theo category, heatmap correlation, seasonal decomposition]

Phụ lục B

Siêu tham số Mô hình

[TT] Thanh Tài

Bảng B.1: Siêu tham số các mô hình

Tham số	RF	RNN	LSTM	CaLSTM
n_steps (look-back)	—	14	14	14
n_features	12	8	10	10
Hidden units	—	64	128	128
n_layers	—	1	2	2
n_estimators	200	—	—	—
Dropout	—	0.2	0.3	0.3
Epochs	—	100	100	100
Batch size	—	64	64	64
Optimizer	—	Adam	Adam	Adam
Learning rate	—	1e-3	1e-3	1e-3
Loss function	MSE	MSE	MSE	Censoring-Aware
Covariate	Có	Có	Có	Có

Phụ lục C

Hướng dẫn Chạy Code

[TT] Thanh Tài

```
1 # To virtual environment
2 python -m venv venv
3 venv\Scripts\activate # Windows
4
5 # C i dependencies
6 pip install pandas pyarrow tensorflow scikit-learn
   statsmodels
7 pip install matplotlib seaborn streamlit fastapi uvicorn
8
9 # C h y EDA
10 python A.M/frn_eda.py
11
12 # C h y training
13 python A.M/frn_train.py --model lstm --impute True
14
15 # C h y dashboard
16 streamlit run A.M/app.py
```

Listing C.1: Cài đặt môi trường